

บทที่ 2

ทฤษฎีที่เกี่ยวข้อง

2.1 Extensible Markup Language (XML)

Extensible Markup Language หรือ XML เป็นมาตรฐานที่องค์กร W3C (World Wide Web Consortium) ได้กำหนดขึ้นเพื่อให้เป็นมาตรฐานใหม่ในการแลกเปลี่ยนข้อมูลระหว่างแพลตฟอร์ม (เป็น Markup Language) โดยเฉพาะการนำไปใช้สำหรับจัดการ แสดงผล และจัดหมวดหมู่ของระบบฐานข้อมูล เมื่อรวมเข้ากับภาษาสำหรับแสดงผล (XSL) และ Document Object Model (DOM) ซึ่งมีมาตรฐานแน่นอน ทำให้ XML เป็นเทคโนโลยีที่ทุกคนสามารถใช้งานได้ทั้งในและนอกอินเทอร์เน็ต

จุดเริ่มต้นของ XML มาจากแนวความคิดเกี่ยวกับรูปแบบของข้อมูลที่สามารถใช้งานได้อย่างกว้างขวาง เหล่าโปรแกรมเมอร์ได้พยายามหาวิธีที่จะแลกเปลี่ยนข้อมูลระหว่างโปรแกรมที่แตกต่างกันมาโดยตลอด ความพยายามขั้นต้นของการรวมความสามารถในการแลกเปลี่ยนข้อมูลและความสามารถในการเก็บข้อมูลเข้าด้วยกันคือ SGML (Standard Generalized Markup Language) ซึ่งเป็นภาษาที่มีมาตรฐานมาจาก Text ซึ่งสามารถใช้กำหนดรูปแบบของข้อมูลได้ (ซึ่งคือ Markup หรือ Metadata¹ นั่นเอง) โดยวิธีการที่เรียกว่า Self Describing

SGML ถูกสร้างขึ้นเพื่อให้เป็นมาตรฐานในการกำหนดรูปแบบของข้อมูลในวัตถุประสงค์ที่หลากหลาย และถูกนำไปใช้อย่างแพร่หลายในระบบการจัดเก็บเอกสารที่มีขนาดใหญ่และซับซ้อน

แอปพลิเคชันที่มีคนรู้จักมากที่สุดของ SGML คือ HyperText Markup Language หรือ HTML ซึ่งเป็นการกำหนดไวยากรณ์ขึ้นมาโดยเฉพาะเพื่อใช้ในการแสดงข้อมูล และเชื่อมโยงส่วนต่างๆ ของ ข้อมูลเข้าไว้ด้วยกัน โดยที่เอกสาร HTML (หรือ webpage) ต้องสามารถแสดงผลได้ในทุก ๆ แอปพลิเคชันที่เข้าใจ HTML นั่นก็คือ เว็บเบราว์เซอร์ และเบราว์เซอร์นั้นต้องสามารถรองรับเอกสารอื่นที่ปลายทาง (หากมีไฮเปอร์ลิงก์เชื่อมโยงไปสู่เอกสารอื่น) ได้ด้วย นอกจากนี้ HTML ซึ่งเป็นไฟล์ประเภท Text ทำให้สามารถสร้างเอกสาร HTML โดยใช้เท็กซ์เอดิเตอร์ทั่ว ๆ ไปหรือใช้เว็บเพจเอดิเตอร์ก็ได้

¹ Metadata หรือ ชุดของข้อมูล คือ โค้ดต่าง ๆ ที่อยู่ในเอกสาร ซึ่งไม่ใช่ตัวข้อมูล แต่มีไว้เพื่ออธิบายลักษณะของข้อมูล เช่น “คำนี้จะต้องเป็นตัวหนา” “ประโยคนี้ต้องอยู่ตรงกลาง” เป็นต้น ซึ่งเมื่อดำด้าเหล่านี้จะทำให้ไฟล์จากต่างโปรแกรมกัน ไม่สามารถเปิดอ่านหรือทำความเข้าใจได้โดยโปรแกรมอื่น ๆ ที่มีการใช้ เมื่อดำด้าที่ต่างกัน

โชคร้ายที่ SGML เป็นภาษาที่มีความซับซ้อนไม่เหมาะกับการนำมาใช้ในกระบวนการแลกเปลี่ยนข้อมูลบนเว็บ และถึงแม้ว่า HTML จะมีผู้นำไปใช้อย่างมากมาย แต่ก็ยังมีข้อจำกัดคือ HTML นั้นถูกสร้างขึ้นมาให้ทำหน้าที่ในการนำเสนอข้อมูลในเบราว์เซอร์เท่านั้น นั่นหมายความว่า หากเราสร้างเอกสาร HTML ขึ้นเพื่อแสดงข้อมูลของใครสักคน HTML ก็จะได้เพียง “แสดง” ข้อมูลเท่านั้น แต่ไม่สามารถที่จะบอกได้ว่าข้อมูลส่วนไหนเป็นข้อมูลที่เกี่ยวข้องกับชื่อของคน ๆ นั้น เนื่องจาก HTML ไม่มีความสามารถที่จะอธิบายข้อมูลพิเศษพวกนั้นได้

Extensible Markup Language (XML) จึงถูกสร้างขึ้นมาจากสาเหตุข้างต้น

XML เป็น subset ของ SGML ซึ่งมีเป้าหมายเดียวกัน (มาร์คอัพข้อมูลทุก ๆ ประเภท) แต่ลดความซับซ้อนลงไปให้มากที่สุดเท่าที่จะเป็นไปได้ XML ถูกออกแบบมาให้เข้ากันได้ทุกประการกับ SGML ซึ่งหมายความว่าเอกสารใด ๆ ก็ตามที่สร้างตามไวยากรณ์ของ XML จะเป็นไปตามไวยากรณ์ของ SGML ด้วย อีกทั้งยังสามารถเปิดได้ด้วยเครื่องมือในการจัดการ SGML ที่มีอยู่แล้ว (แต่เอกสารของ SGML ไม่จำเป็นต้องเป็นเอกสาร XML เสมอไป)

โดยความเป็นจริงแล้ว XML ไม่ได้เป็น “ภาษา” เสียทีเดียว แต่เป็นมาตรฐานของการสร้างภาษา หรือเป็นตัวกำหนดไวยากรณ์ที่คุณใช้สำหรับสร้างภาษาของตนเอง

2.2 ความแตกต่างของ XML และ HTML

หาก HTML สร้างขึ้นเพื่อการแสดงผล XML ก็ถูกสร้างมาเพื่อใช้ในการแลกเปลี่ยนข้อมูลนั่นเอง โดยข้อแตกต่างที่สำคัญระหว่าง HTML และ XML คือ

HTML ถูกสร้างมาสำหรับแอปพลิเคชันเฉพาะ เพื่อแสดงข้อมูลแก่มนุษย์ ซึ่งมีกลุ่ม Tag ที่แน่นอนอยู่กลุ่มหนึ่ง (เช่น <P>, , <H2> เป็นต้น) เพื่อใช้สำหรับสร้างเอกสาร HTML ซึ่งไม่ได้ช่วยสื่อความหมายหรืออธิบายข้อมูลที่อยู่ใน Tag นั้นเลย และในทางทฤษฎี เราคงคิดว่า web browser ต่าง ๆ คงเข้าใจและแสดงผลเอกสาร HTML ออกมาได้เหมือน ๆ กัน แต่ในทางปฏิบัติแล้วเราพบว่า webpage บางอันแสดงผลผิดเพี้ยนไปจากที่ตั้งใจบนบาง web browser ซึ่งแสดงว่า web browser แต่ละตัวมีความสามารถในการแปล Tag ต่าง ๆ ได้ไม่เท่ากัน นอกจากนี้ การที่ W3C เป็นผู้กำหนดความหมายให้กับ Tag ของ HTML ทำให้ผู้พัฒนา web รวมไปถึงผู้พัฒนา browser ก็ต้องคอยติดตามเรียนรู้ Tag เหล่านั้น เมื่อผู้พัฒนา web อยากให้มีอะไรใหม่ ๆ เพิ่มเติม ก็ต้องทำการร้องขอไปยังองค์กรดังกล่าวให้อิมพลีเมนต์ Tag นั้นเพิ่ม ซึ่งก็ไม่เพียงพออยู่ดี และยังทำให้เกิดปัญหาที่บาง web browser ยังไม่รองรับ Tag ใหม่ ๆ และแสดงผลไม่ถูกต้องอีกด้วย ทำให้ HTML ขาดความยืดหยุ่นในส่วนนี้ไป

แต่ XML สามารถกำหนด Tag ขึ้นเองได้ โดย Tag ดังกล่าวจะเป็นส่วนที่อธิบายข้อมูลที่อยู่ระหว่าง Tag นั้น ทำให้ XML มีความยืดหยุ่นสูงและช่วยให้สามารถอธิบายข้อมูลด้วยตัวเองได้ ซึ่ง HTML ไม่สามารถทำได้ และเนื่องจาก XML แยกส่วนของข้อมูลออกจากส่วนของการแสดงผล ทำให้ XML ไม่ขึ้นอยู่กับแอปพลิเคชันใด ๆ มันสามารถใช้งานกับแอปพลิเคชันใด ๆ ก็ได้ ที่คุณต้องการใช้งาน และทำให้ผู้เขียนเอกสาร XML ทำงานกับข้อมูลได้อย่างง่ายดายภายใต้แอปพลิเคชันต่าง ๆ รวมถึงการแบ่งปันข้อมูลกับผู้อื่นในอินเทอร์เน็ตได้อย่างง่ายดาย โดยเราสามารถส่งข้อมูลของ <name> (อิลิเมนต์ที่เรากำหนดเอง) ไปทั่วอินเทอร์เน็ตโดยที่ไม่จำเป็นต้องมีโปรแกรมเฉพาะสำหรับใช้งานกับข้อมูลชุดนี้ คนอื่น ๆ จะสามารถเปิดไฟล์ของ <name> ด้วยเท็กซ์เอดิเตอร์ธรรมดาเพื่อที่จะดูโค้ด XML หรือจะใช้โปรแกรมสำหรับดูผลลัพธ์ (viewer) เช่น Internet Explorer (IE5 ขึ้นไป) ก็ได้ หรือหากเราต้องการกำหนดโครงสร้างของข้อมูลที่จะมีโปรแกรมที่ทำงานร่วมเพียงโปรแกรมเดียวก็สามารถทำได้เช่นกัน และสามารถจะแบ่งปันข้อมูลของเรากับโปรแกรมอื่นหรือแม้แต่บริษัทอื่น ๆ ในอินเทอร์เน็ต XML ก็สามารถทำได้ หรือถ้าต้องการให้เอกสาร XML แสดงผลออกมาอย่างสวยงามในรูปแบบของ webpage ก็สามารถเลือกใช้เอกสาร XSL ร่วมกับเอกสาร XML เพื่อแปลงข้อมูลที่เก็บเป็น XML ให้แสดงผลออกมาในรูปแบบ (X)HTML บน web browser ได้

คำว่า Extensible ใน Extensible Markup Language มีที่มาจากการที่ทุกคนสามารถจะกำหนดรูปแบบและโครงสร้างของข้อมูลได้อย่างอิสระตามที่ต้องการโดยใช้ XML และคนอื่น ๆ อาจจะกำหนดข้อมูลชุดเดียวกันนี้ในรูปแบบที่ต่างกันอย่างสิ้นเชิงก็ตาม ซึ่งต่างจากโปรแกรมจัดการฐานข้อมูล หรือวิธีการจัดการฐานข้อมูลแบบเก่าที่ข้อมูลชุดเดียวกันมักมีโครงสร้างที่คล้ายกัน ไม่ค่อยแตกต่างกันมากนัก

และเนื่องจากความยืดหยุ่นในการใช้งานค่อนข้างสูงของ XML นี้ ทำให้มันถูกวางให้เป็นมาตรฐานของการแลกเปลี่ยนข้อมูล

ตัวอย่างที่ 2.1 – 2.3 แสดงให้เห็นลักษณะของเอกสารข้อมูลเกี่ยวกับชื่อแบบ (2.1) Text File, (2.2) HTML และ (2.3) XML

ตัวอย่างที่ 2.1 แสดง “name.txt” ซึ่งเป็น Text File ของข้อมูล

John Doe

ตัวอย่างที่ 2.2 แสดง “name.html” โดยเขียนข้อมูลในรูปแบบ HTML

```
<HTML>
<HEAD><TITLE>Name</TITLE></HEAD>
<BODY>
  <P>John Doe</P>
</BODY>
</HTML>
```

ตัวอย่างที่ 2.3 แสดง “name.xml” บรรจุข้อมูลที่อธิบายด้วย XML

```
<name>
  <first>John</first>
  <last>Doe</last>
</name>
```

จากตัวอย่างข้างต้น จะทำให้เราเข้าใจได้ว่า เหตุใดภาษาย่าง SGML หรือ XML จึงถูกเรียกว่า Self Describing (อธิบายตัวเอง) ตัวอย่างที่ 2.3 จะเห็นได้ว่าเป็นข้อมูลเกี่ยวกับ <name> และยังมีข้อมูลส่วนที่กำหนดให้เป็น <first> และส่วนที่เป็น <last> อันที่จริงแล้ว เราสามารถกำหนดข้อมูลให้มีชื่อเรียกเป็นอะไรก็ได้ตามแต่ต้องการ (แตกต่างกับ HTML ซึ่งจะมีกลุ่มของ tag ที่แน่นอนเพื่อใช้ในการแสดงผล เช่น <P>, <H1>, เป็นต้น) แต่เพื่อให้บรรลุประโยชน์จาก XML เราควรกำหนดชื่อต่าง ๆ ให้สื่อความหมายเกี่ยวกับข้อมูลที่บรรจุอยู่ด้วย

2.3 เราจะเรียกใช้ข้อมูลที่เขียนด้วย XML ได้อย่างไร?

โปรแกรมประเภท “Parsers” คือโปรแกรมที่สามารถอ่านไวยากรณ์ของ XML และดึงข้อมูลที่เราต้องการมาให้เราได้โดยอัตโนมัติ เพียงแค่เราจัดโครงสร้างข้อมูลให้อยู่ในรูปแบบของ XML เราสามารถส่งไฟล์ (ไฟล์ XML ใช้นามสกุล .xml) ดังตัวอย่างที่ 2.4 ไปให้ XML parsers ได้เลย

ตัวอย่างที่ 2.4 ตัวอย่างไฟล์เอกสาร XML

```
<name>
  <first>John</first>
  <middle>Fitzgerald Johansen</middle>
  <last>Doe</last>
</name>
```

XML Parser คือตัวแปลความหมาย XML หรืออีกชื่อหนึ่งคือ XML Processor ถือว่าเป็นสิ่งที่มีบทบาทสำคัญมาก เพราะเป็นตัวแปลเอกสาร XML หน้าทีพื้นฐานของ XML Parser ก็คือ อ่าน (Read), สร้าง (Create), แก้ไข (Update) และเข้าถึง (Manipulate) เอกสาร XML

จากตัวอย่างที่ 2.4 Parser จะสามารถบอกเราได้ทันทีว่ามีส่วนของข้อมูลที่มีชื่อว่า <middle> อยู่ และข้อมูลที่เก็บอยู่ในส่วนนั้นก็คือ Fitzgerald Johansen² โดยผู้ที่พัฒนาโปรแกรม parser ไม่จำเป็นต้องทราบกฎเกณฑ์ต่าง ๆ เลยว่าตรงไหนเป็นจุดจบของชื่อ ตรงไหนเป็นจุดเริ่มต้นของชื่อกลาง รวมถึงไม่จำเป็นต้องทราบ เกี่ยวกับแอปพลิเคชันของผู้พัฒนา หรือ <name> เลย นอกจากนี้ parser ตัวเดียวกันนี้ นอกจากจะใช้แอปพลิเคชันของผู้พัฒนา ยังสามารถนำไปใช้กับแอปพลิเคชันอื่น ๆ ที่อาจแตกต่างกันอย่างสิ้นเชิงเลย ก็ได้ และยังสามารถใช้ภาษาใด ๆ ก็ได้โดยไม่เกี่ยวข้องกับ parser โดยที่ทุก ๆ ภาษาสามารถทำงานร่วมกันได้กับ parser ตัวเดียวกันได้ แม้ว่าตัวผู้พัฒนา parser นั้น ๆ จะไม่เข้าใจภาษาที่นำมาใช้ใน XML นั้น ๆ เลยก็ตาม นอกจากนี้เรายังสามารถเพิ่มเติมฟอร์แมตของ XML โดยไม่จำเป็นต้องไปยุ่งกับ code เก่า ๆ และแอปพลิเคชันใหม่ยังสามารถใช้ประโยชน์จากข้อมูลใหม่ที่เพิ่มเข้ามาได้ด้วย (นอกเสียจากว่าจะมีการตัดทอนบางอิลิเมนต์ของ <name> ออก หรือมีการเปลี่ยนแปลงชื่อของอิลิเมนต์ นั้นจะทำให้เราจำเป็นต้องมีการแก้ไขแอปพลิเคชันเพื่อให้สอดคล้องกับความเปลี่ยนแปลงที่เกิดขึ้น)

² หากเราสร้างเอกสาร XML ขึ้นมาสักไฟล์หนึ่ง เราสามารถมั่นใจได้ว่า ไม่ว่า XML parser ตัวใด ๆ ก็สามารถนำข้อมูลจากเอกสารที่เราสร้างมาใช้ได้อย่างแน่นอน แต่ไม่อาจรับประกันได้ว่าจะมีแอปพลิเคชันใด ๆ ที่สามารถเข้าใจความหมายที่แท้จริงของข้อมูลเหล่านั้น นั่นคือ parser จะสามารถบอกได้ว่าส่วนไหนของข้อมูลที่มีชื่อว่า <middle> และข้อมูลที่อยู่ในนั้นคือ Fitzgerald Johansen แต่ไม่ได้หมายความว่า จะมีแอปพลิเคชันที่เข้าใจว่า <middle> คืออะไร หรือจะนำไปใช้ประโยชน์ได้อย่างไร ดังนั้นเราอาจสร้างเอกสาร XML มาเพื่ออธิบายข้อมูลที่เราต้องการได้ แต่หากจะให้มันเกิดประโยชน์จริง ๆ แล้ว จำเป็นต้อง มีแอปพลิเคชันซึ่งสามารถเข้าใจ XML ที่เราสร้างขึ้นมามีด้วย

ในปัจจุบันมีบริษัทซอฟต์แวร์หลายรายผลิตโปรแกรม XML Parser มากมาย สร้างขึ้นจากภาษาต่าง ๆ ที่พบมากคือ C, C++, Java, Perl ยกตัวอย่างเช่น

1. MSXML ของไมโครซอฟต์ เป็นโปรแกรมที่พ่วงมากับ IE 5.0 ขึ้นไป
2. เทคโนโลยี COM (Component Object Model) ของไมโครซอฟต์
3. XML4J (XML for Java) เป็นผลิตภัณฑ์ของไอบีเอ็ม สร้างด้วยภาษาจาวา
4. JAXP (Java API for XML Parsing) เป็นผลิตภัณฑ์ของซัน สร้างโดยภาษาจาวา
5. DataChannel XJParser ของ DataChannel, Inc. สร้างด้วยภาษาจาวา
6. XP เขียนด้วยภาษาจาวา เป็นผลงานของ Mr. James Clark
7. XTเขียนด้วยภาษาซี ของ Mr. James Clark
8. Expatเขียนด้วยภาษาซี ของ Mr. James Clark
9. Xerces-J เขียนด้วยภาษาจาวา ของ Apache XML Project
10. Xerces-C++ เขียนด้วยภาษา C++ เป็นผลงานของ Apache XML Project

การจำแนกชนิดของ XML Parser แบ่งได้หลายวิธี แต่หลักเกณฑ์ที่นิยมมี 2วิธี คือ

1. XML Parser แบบ Validating และแบบ Non-Validating
2. XML Parser ที่รองรับ DOM (Document Object Model) และ SAX (Simple API for XML)

สำหรับ XML Parser แบบ Validating หมายถึง XML Parser ที่มีการตรวจสอบคุณสมบัติ Well-Formed และคุณสมบัติ Valid ของเอกสาร XML Parser แบบ Non-Validation หมายถึง XML Parser ที่ไม่ตรวจสอบคุณสมบัติ Valid และสนใจเฉพาะคุณสมบัติ Well-Formed

โปรแกรม XML Parser ส่วนใหญ่เป็นแบบ Non-validating เพราะสร้างขึ้นได้ง่ายกว่า

แต่ถ้าจำแนกประเภทของ XML Parser ตามวิธีการสำรวจเนื้อหาเอกสาร XML ก็แบ่งได้เป็น XML Parser แบบที่รองรับ DOM และแบบที่รองรับ SAX โดยแบบที่รองรับ DOM จะมองเอกสาร XML เป็นโครงสร้างต้นไม้ (Tree-Based Parser) และแบบที่รองรับ SAX จะใช้วิธีผูกอิเลเมนต์ในเอกสารเข้ากับเหตุการณ์ และใช้วิธีสำรวจเนื้อหาเอกสารตามเหตุการณ์เงื่อนไขที่ระบุ (Event-Driven Parser)

2.4 อะไรคือส่วนประกอบในการสร้าง XML

ข้อกำหนดที่ใช้ในการทำงานควบคุมกันในกลุ่มของ XML ที่สำคัญบางเรื่อง ได้แก่

2.4.1 XML 1.0 เป็นข้อกำหนดพื้นฐาน ซึ่งเกี่ยวข้องกับการสร้างกลุ่มของเทคโนโลยี XML กล่าวถึงไวยากรณ์ซึ่งเอกสาร XML ต้องยึดถือ และกฎเกณฑ์ที่ XML parser จำต้องยึดถือ รวมถึงทุก ๆ สิ่งที่เป็นต่อการอ่านหรือเขียนเอกสาร XML

2.4.2 เนื่องจากเราสามารถกำหนดโครงสร้างและชื่อของอิลิเมนต์ขึ้นมาสำหรับใช้ในเอกสารของเราได้ จึงจำเป็นต้องมี DTD และ **สกีมา (Schemas)** เพื่อช่วยในการจัดสร้างเว็บเพจสำหรับประเภทของเอกสารขึ้นมา เพื่อตรวจสอบว่าเอกสารต่าง ๆ เป็นไปตามที่กำหนดไว้ในเว็บเพจ และนักพัฒนาคนอื่น ๆ จะสามารถสร้างเอกสารที่เข้ากันได้กับเว็บเพจนั้น ๆ

2.4.3 **เนมสเปซ (Namespace)** ช่วยให้เราสามารถแยกแยะ XML vocabulary แต่ละอันออกจากกันได้ ทำให้เราสามารถสร้างเอกสารที่ใช้งานได้กว้างขึ้น

2.4.4 **XPath** ใช้กำหนดรายละเอียดของภาษาที่ใช้ในการสืบค้นข้อมูลบางส่วนของเอกสาร XML ทำให้แอปพลิเคชันต่าง ๆ สามารถค้นหาข้อมูลบางส่วนของเอกสาร XML ที่ต้องการได้

2.4.5 ในกรณีที่ต้องการแสดงผลเอกสาร XML ที่ไม่ซับซ้อน สามารถใช้ **Cascading Style Sheets (CSS)** เพื่อแสดงผลเอกสารได้ และถ้าเอกสารนั้นมีความซับซ้อนเราก็สามารถใช้ **Extensible Style sheet Language (XSL)** ที่เกิดมาจาก **XSLT** ซึ่งสามารถแปลงเอกสารจากประเภทหนึ่งไปสู่อีกประเภทหนึ่งได้ รวมถึง **Formatting Object** ที่ทำงานเกี่ยวกับการแสดงผลเช่นกัน

2.4.6 **XLink** และ **XPointer** เป็นภาษาที่ใช้สำหรับเชื่อมโยงเอกสาร XML เข้าไว้ด้วยกัน ในลักษณะที่ใกล้เคียงกับไฮเปอร์ลิงค์ในเอกสาร HTML

2.4.7 **Document Object Model (DOM)** เป็นแอปพลิเคชันที่มักใช้ในการติดต่อสื่อสารกับเอกสาร XML กันอย่างแพร่หลาย และ **Simple API for XML (SAX)** ซึ่งเป็นอีกทางเลือกที่โปรแกรมเมอร์ใช้ติดต่อกับเอกสาร XML จากโค้ดของ XML เอง

2.5 การสร้างเอกสาร Well-Formed XML

Well-Formed XML หรือ XML **ที่ลงตัว** คือ XML ที่เป็นไปตามกฎเกณฑ์ทั่ว ๆ ไปของ XML 1.0 ซึ่งมีกฎเกณฑ์พื้นฐานในการเขียนเอกสาร XML ที่ควรทราบดังนี้

2.5.1 Tag, Text และ Element

พิจารณาส่วนประกอบต่าง ๆ ที่สำคัญในเอกสาร XML จากตัวอย่างที่ 2.5

ตัวอย่างที่ 2.5 ส่วนประกอบสำคัญสำหรับ XML

```
<name>
  <first>John</first>
  <middle>Fitzgerald Johansen</middle>
  <last>Doe</last>
</name>
```

คำที่อยู่ระหว่าง < และ > เรียกว่า **แท็ก (Tag)** สำหรับ XML ข้อมูลในเอกสารของเราจะอยู่ภายในแท็กหลาย ๆ ตัวที่ประกอบเป็นมาร์คอัพของเอกสาร ซึ่งวิธีนี้จะช่วยให้ง่ายในการแยกระหว่างข้อมูล(information) และมาร์คอัพ

จากตัวอย่างจะเห็นได้ว่า แท็กต่าง ๆ จะอยู่กันเป็นคู่ ๆ โดยเมื่อมีแท็กเริ่มต้นก็ต้องมีแท็กสิ้นสุดด้วย โดยใน XML จะเรียกแท็กเริ่มต้นว่า **start-tags** และแท็กสิ้นสุดคือ **end-tags** โดยที่แท็กสิ้นสุดจะเหมือนกับแท็กเริ่มต้นทุกประการ แต่จะมีเครื่องหมาย “/” อยู่ทางด้านขวาของ “<” เท่านั้น (เหมือนแท็กเริ่มต้นและแท็กสิ้นสุดของ HTML)

สำหรับสิ่งต่าง ๆ ทั้งหมด นับตั้งแต่จุดเริ่มต้นของแท็กเริ่มต้น ไปจนถึงจุดสิ้นสุดของแท็กสิ้นสุด รวมทั้งสิ่งที่อยู่ระหว่างแท็กทั้งสอง เรียกว่า **อิลิเมนต์ (Element)**

ดังนั้น จากตัวอย่างที่ 2.5

<first> เป็นแท็กเริ่มต้น

</first> เป็นแท็กสิ้นสุด

<first>John</first> เป็นอิลิเมนต์

สิ่งที่อยู่ระหว่างแท็กเริ่มต้นและแท็กสิ้นสุดของแต่ละอิลิเมนต์เรียกว่า **เนื้อหาของอิลิเมนต์ (Text)** ซึ่งมักจะเป็นข้อมูลอย่างเดียว จากตัวอย่างที่ 2.5 เนื้อหาของอิลิเมนต์ของเรา เป็นเพียงข้อมูลประเภทตัวหนังสือ (Persed Character DATA) ซึ่งมักเรียกว่า **PCDATA** (อ่านว่าพีซีดาต้า)

2.5.2 ลำดับชั้นใน XML

XML มีการจัดแบ่งข้อมูลเป็นลำดับชั้น โดยอาศัยความสัมพันธ์แบบ parent/child และ sibling/sibling โดยความสัมพันธ์ต่าง ๆ สามารถอธิบายได้โดยเปรียบเทียบกับ **Family Tree**

อิลิเมนต์ลำดับสูงสุดในเอกสาร เรียกว่า **รูทอิลิเมนต์ (Root Element)**

สำหรับลูกหลานของอิลิเมนต์นั้นเรียกว่า content โดยแบ่งเป็น

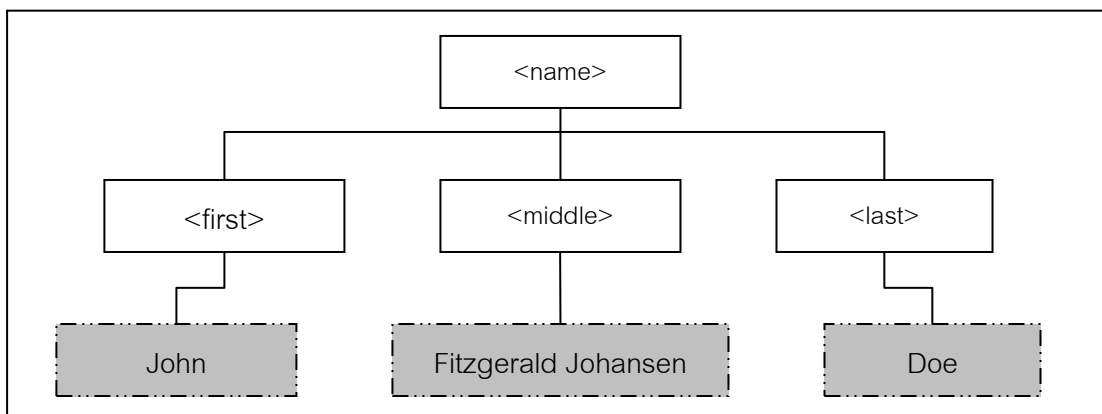
Element content ใช้เรียกลูกหลานของอิลิเมนต์ที่ประกอบด้วย อิลิเมนต์เท่านั้น

Simple content ใช้เรียกลูกหลานของอิลิเมนต์ที่ประกอบด้วย ข้อความ (PCDATA) เท่านั้น

Mix content ใช้เรียกลูกหลานของอิลิเมนต์ที่ประกอบด้วย อิลิเมนต์หรือข้อความก็ได้

จากตัวอย่าง 2.5 <name> คือ Root Element ซึ่งสามารถเขียนลำดับชั้นได้ดังแสดงในตัวอย่างที่ 2.6

ตัวอย่างที่ 2.6 ลำดับชั้นของข้อมูล XML ในตัวอย่างที่ 2.5



2.5.3 กฎของอิลิเมนต์

ในการสร้างเอกสาร XML จะต้องยึดกฎเหล่านี้อย่างเหนียวแน่นเพื่อให้เอกสารนั้นมีความสมบูรณ์ได้แก่

1. ทุก ๆ start-tags จะต้องมี end-tags ที่เข้าคู่กันเสมอ
2. tag ต่าง ๆ จะคาบเกี่ยวกันไม่ได้ (ต้องนำมาเขียนในรูปแบบของ tree ที่ถูกต้องได้เสมอ)
3. เอกสาร XML จะมี Root Element ได้เพียงหนึ่งเดียวเท่านั้น (ถ้าสร้างแล้วมีหลาย Root ให้เพิ่ม Root สูงสุดเข้าไปบนชั้นที่เหนือกว่า)
4. แม้ว่า XML จะไม่มีการกำหนดคำสงวน (Reserved Word) เอาไว้ ทำให้เราสามารถตั้งชื่อของอิลิเมนต์ได้อย่างหลากหลาย แต่ก็ยังมีกฎของการกำหนดชื่อที่เราต้องปฏิบัติตามดังนี้

4.1 ชื่อของอิลิเมนต์ต้องขึ้นด้วย ตัวอักษร (รวมทั้งอักขระที่ไม่ใช่ลาติน) หรือ “_” แต่ห้ามใช้ ตัวเลข หรือเครื่องหมายวรรคตอนอื่น ๆ ขึ้นต้นชื่อ

4.2 หลังจากอักขระตัวแรกสามารถใช้ตัวเลข, “-”, “.” ได้

4.3 ชื่อของอิลิเมนต์ต้องไม่มีช่องว่าง

4.4 ควรใช้ “:” สำหรับกรณีเป็น namespace เท่านั้น

4.5 ไม่สามารถตั้งชื่อที่เริ่มด้วยตัวอักษร “xml” ไม่ว่าจะเป็นตัวพิมพ์ใหญ่ พิมพ์เล็กหรือผสมกัน เช่น “xml”, “XML”, “xMI”, “Xml” หรือการผสมกันในแบบอื่น ๆ ไม่สามารถใช้ได้

4.6 ห้ามใช้ “=” ในการตั้งชื่อ (ทั้ง ๆ ที่ไม่ใช่แอตทริบิวต์) เช่น

<fun=xml> ใช้ไม่ได้

<name nickname='Shiny John'> ใช้ได้ เพราะชื่อของอิลิเมนต์คือ name ซึ่งเป็นไปตามกฎ ส่วน nickname='Shiny John' เป็นการกำหนดแอตทริบิวต์ (จะพูดถึงในหัวข้อต่อไป)

4.7 จะต้องไม่มีช่องว่างต่อจาก "<" แต่สามารถมีช่องว่างก่อนถึง ">" ได้

<name> หรือ <name > สามารถใช้ได้

< name> ใช้ไม่ได้ เพราะมีช่องว่างต่อจาก "<"

4.8 ชื่อของอิลิเมนต์ ห้ามมีการเว้นวรรคในชื่อ เช่น <my tag> ใช้ไม่ได้

5. XML ให้ความสำคัญกับตัวอักษรเล็ก – ใหญ่ (Case Sensitive) หมายถึง คำเดียวกันแต่พิมพ์ด้วยตัวพิมพ์เล็ก – ใหญ่ที่ต่างกัน จะถือว่าแตกต่างกัน

เช่น <name> กับ <Name> ถือว่าเป็นคนละอิลิเมนต์กัน ดังนั้น ในการเขียนเอกสาร XML เราควรเลือกรูปแบบของการกำหนดชื่อขึ้นมาใช้ประจำสักหนึ่งรูปแบบ เพื่อป้องกันปัญหาที่เกิดจากกรณีดังกล่าว

6. XML จะไม่ตัดส่วนที่เป็น white space (การเว้นวรรค, ย่อหน้า, enter) ในข้อความ (PCDATA) ออก

ซึ่งแตกต่างกับ HTML ที่ช่องว่างที่ไม่มีความหมายจะถูกตัดออกจากเอกสารขณะประมวลผล

2.5.4 การกำหนดแอตตริบิวต์ ลงในแท็กและอิลิเมนต์

แอตตริบิวต์ (Attribute) คือ คู่ของชื่อและค่า ที่สัมพันธ์กันและสัมพันธ์กับอิลิเมนต์นั้น ๆ ด้วย ซึ่งอยู่ในรูปแบบ ชื่อ='ค่า'

การเขียนแอตตริบิวต์จะเขียนเพิ่มลงไปบริเวณแท็กเริ่มต้น โดยทุก ๆ แอตตริบิวต์ต้องมีค่าของแอตตริบิวต์อยู่เสมอ ซึ่งค่าของแอตตริบิวต์ต้องอยู่ภายในเครื่องหมายคำพูด จะเป็น double quote (") หรือ single quote (') ก็ได้ แต่ควรเลือกแบบใดแบบหนึ่งเป็นแบบที่ใช้ประจำ (และถ้าผสมเป็น " ' ไม่สามารถใช้ได้) และแม้ว่าค่าของแอตตริบิวต์จะเป็นเพียงสตริงว่าง ๆ เช่น "" ก็ต้องใส่ดังตัวอย่างที่ 2.7 แสดงถึงตัวอย่างการกำหนดแอตตริบิวต์ที่ถูกต้อง

ตัวอย่างที่ 2.7 การกำหนดแอตตริบิวต์

```
<name nickname='Shiny John'>
  <first>John</first>
  <middle>Fitzgerald Johansen</middle>
  <last>Doe</last>
</name>
```

สำหรับกฎการตั้งชื่อแอตทริบิวต์นั้นเหมือนกับกฎการตั้งชื่ออิลิเมนต์

ในอิลิเมนต์หนึ่ง สามารถมีแอตทริบิวต์ได้หลาย ตัว แต่ไม่สามารถกำหนดให้มีแอตทริบิวต์มากกว่า 1 ตัวที่มีชื่อเดียวกันอยู่ภายในอิลิเมนต์เดียวกันได้ เช่น `<bad att='1' att='2'></bad>` ใช้ไม่ได้ แต่ถ้าเป็นคนละชื่อกัน เช่น `<good att1='1' att2='2'></good>` สามารถใช้ได้

ทำไมต้องใช้แอตทริบิวต์?

ในความเป็นจริงแล้ว ไม่มีความแตกต่างกันเลยระหว่างตัวอย่างที่ 2.8 และ 2.9

ตัวอย่างที่ 2.8 การอธิบายข้อมูลด้วยการใช้แอตทริบิวต์

```
<name nickname='Shiny John'></name>
```

ตัวอย่างที่ 2.9 การอธิบายข้อมูลด้วยการใช้อิลิเมนต์

```
<name>
  <nickname >Shiny John</nickname >
</name>
```

แต่ข้อดีที่ทำให้มีการใช้แอตทริบิวต์กับค่าบางค่า เนื่องจาก แอตทริบิวต์สามารถกำหนด Metadata ที่อาจไม่เป็นที่ต้องการของบางแอปพลิเคชัน โดยไม่มีผลกระทบไปถึง XML ตัวอย่างเช่น เอกสาร XML ที่ใช้ในการเก็บข้อมูลของรายการหนังสือหนึ่งถูกนำไปใช้กับหลาย ๆ ห้องสมุด (หลายแอปพลิเคชัน) แต่มีบางแอปพลิเคชันต้องการใช้ข้อมูลของ serial number ของหนังสือ ในขณะที่แอปพลิเคชันอื่น ๆ ไม่ต้องการ เราสามารถให้ serial number นั้นกำหนดเป็นแอตทริบิวต์ เพื่อแยกข้อมูลที่แอปพลิเคชันส่วนมากต้องการออกจากข้อมูลที่มีบางแอปพลิเคชันเท่านั้นที่จะใช้ได้

2.5.5 คอมเมนต์ (Comment)

คอมเมนต์ในภาษา XML มีประโยชน์ในการใช้งานเพื่ออธิบายโค้ด XML เพื่อที่จะต้องกลับมาแก้ไขภายหลัง หรือเพื่อให้ผู้อื่นที่ต้องการพัฒนาต่อสามารถเข้าใจโค้ดที่เราเขียนไว้เดิมได้

- คอมเมนต์จะเริ่มต้นด้วย `<!--` และจบด้วย `-->` เสมอ
- ไม่สามารถแทรกคอมเมนต์ลงไปภายใน tag ได้
- ไม่สามารถใช้ “-” ภายในคอมเมนต์ได้

ตัวอย่างที่ 2.10 แสดงการใช้คอมเมนต์ที่ถูกต้อง

```
<name nickname='Shiny John'>
  <first>John</first>
  <!--John lost his middle name in a fire-->
  <middle></middle>
  <last>Doe</last>
</name>
```

ตัวอย่างที่ 2.11 การใช้คอมเมนต์ที่ไม่ถูกต้อง เนื่องจากแทรกคอมเมนต์ใน tag

```
<middle></middle <!--John lost his middle name in a fire--> >
```

ตัวอย่างที่ 2.12 การใช้คอมเมนต์ที่ไม่ถูกต้อง เนื่องจากมีการใช้ “-“ ภายในคอมเมนต์

```
<!--John lost his middle name -- in a fire-->
```

XML parser มักจะไม่ส่งคอมเมนต์ต่าง ๆ ไปยังแอปพลิเคชัน หมายความว่า ผู้อ่านจะไม่สามารถสั่งให้แอปพลิเคชันต่าง ๆ ใช้ข้อมูลที่อยู่ภายในคอมเมนต์ได้

เนื่องจากคอมเมนต์จะไม่ถูกส่งไปยังแอปพลิเคชัน ดังนั้น หากมีข้อความใด ๆ ที่จำเป็นต้องส่งไปให้แอปพลิเคชันใช้งาน ต้องนำไปใส่ไว้ในอีลิเมนต์หรือแอตตริบิวต์

2.5.6 อีลิเมนต์ว่าง (Empty Element)

อีลิเมนต์ว่าง คือ อีลิเมนต์ที่ไม่มีข้อมูลอยู่ภายใน ใช้เมื่อ อีลิเมนต์นั้นไม่มี Metadata (ซึ่งในชุดอื่น ๆ อาจมีเมตาดาต้า หรือสร้างเพื่อไว้รอการเพิ่มข้อมูลในภายหลังก็ได้)

จากตัวอย่างที่ 2.10 อีลิเมนต์ว่างคือ `<middle></middle>` หรืออาจเขียนว่า `<middle/>` ซึ่งไม่ต้องมี tag สิ้นสุดก็ได้ (มีความหมายเหมือนกัน)

ในการเขียนอีลิเมนต์ว่างแบบหลังนั้น เราสามารถเว้นวรรคก่อน `</>` ได้ แต่เว้นวรรคที่อื่นไม่ได้ เช่น

`<middle />` ถูกต้อง

แต่

`<middle/ >`
`<middle / >` } ไม่ถูกต้อง

2.5.7 การประกาศว่าเป็น XML

บ่อยครั้งที่การแยกแยะเอกสารว่าเป็นประเภทไหน ช่วยให้เราทำงานได้ง่ายขึ้นมาก ดังนั้น XML จึงกำหนดให้มีวิธีการประกาศให้เป็น XML (XML Declaration) ให้กับผู้ใช้งาน เพื่อกำหนดประเภทของเอกสารให้เป็น XML พร้อมกับเพิ่มข้อมูลอีกนิดหน่อยให้กับ XML parser

ตามปกติการประกาศประเภทของเอกสารไม่มีความจำเป็นที่จะต้องใส่ แต่อย่างไรก็ตาม ในการเขียนเอกสาร XML เราก็ควรจะใส่มันเอาไว้ด้วย

การประกาศประเภทของเอกสารแบบธรรมดา จะมีลักษณะดังตัวอย่างที่ 2.13

ตัวอย่างที่ 2.13 แสดงการประกาศประเภทของเอกสาร XML

```
<?xml version='1.0' encoding='UTF-16' standalone='yes'?>
<name nickname='Shiny John'>
  <first>John</first>
  <!--John lost his middle name in a fire-->
  <middle/>
  <last>Doe</last>
</name>
```

ข้อกำหนดบางประการที่จำเป็นต้องจำในการประกาศประเภทของเอกสาร XML

- การประกาศประเภทของเอกสารจะเริ่มต้นด้วยอักขระ **<?xml** และจบด้วย **?>**
- ถ้าหากกำหนดประเภทของเอกสารลงไปแล้ว จำเป็นต้องมีการกำหนด version ด้วย แต่สำหรับ encoding และ standalone ถือว่าเป็นส่วนขยาย ซึ่งจะใส่หรือไม่ก็ได้
- การเรียงลำดับ version, encoding³ และ standalone⁴ ต้องเรียงให้เป็นไปตามลำดับ ดังตัวอย่างที่ 2.13
- หมายเลข version⁵ เป็นตัวบอกว่ามาตรฐานใดของ XML ที่เอกสารนี้อ้างอิงอยู่ หากตัวเลข version นั้นสูงกว่า version ที่กำหนดให้ จะถูกยกเลิกการทำงานกับเอกสาร

³ แอตทริบิวต์ encoding เป็นการแจ้งว่า การเข้ารหัสตัวอักษรแบบใดที่เราใช้อยู่ในเอกสารนี้ หากไม่ได้ระบุ XML parser จะเลือกใช้ UTF-8 หรือ UTF-16 ซึ่งเป็นมาตรฐานตามข้อกำหนดโดยอัตโนมัติ

⁴ ค่าของแอตทริบิวต์ Standalone มีได้เพียง 2 ค่าคือ Yes และ No โดย Yes จะบอกให้ parser รู้ว่าเอกสารนี้สมบูรณ์ในตัวเองแล้ว ไม่จำเป็นต้องอาศัยเอกสารอื่นมาเสริมในการทำงาน และ No บอกให้ทราบว่า เอกสารนี้ต้องทำงานร่วมกับเอกสารอื่น

- การประกาศประเภทของเอกสารจะต้องอยู่ในตำแหน่งแรกของไฟล์ และด้วยข้อกำหนดข้างต้น ตัวอักษรตัวแรกของไฟล์จึงต้องเป็น "<" และห้ามมีการเว้นวรรคหรือเว้นช่องว่าง ถึงแม้ว่าในบาง parser อาจไม่เข้มงวดนักแต่จะเป็นการดี ถ้าเราจะทำให้ถูกต้องไว้ก่อน

จากข้อกำหนดต่าง ๆ ข้างต้น การประกาศประเภทของเอกสารอาจจะเต็มรูปแบบดังตัวอย่างที่ 2.13 หรือเป็นดังตัวอย่างที่ 2.14 ก็ได้

ตัวอย่างที่ 2.14 การประกาศประเภทของเอกสาร

```
<?xml version='1.0' ?>
```

2.5.8 ตัวอักษรต้องห้ามใน PCDATA

มีบางตัวอักษรที่ผู้ใช้ไม่สามารถใช้ใน PCDATA ได้ เนื่องจากว่าเป็นไวยากรณ์ของ XML ได้แก่ &, <, >, ' , "

สำหรับทางแก้สำหรับผู้ที่ต้องการใช้ตัวอักษรดังกล่าว มี 2 วิธี ได้แก่

1. ใช้รหัสที่เรียกว่า Entity Reference แทนตัวอักษรที่ห้ามใช้ ดังตารางที่ 2.1

ตารางที่ 2.1 การใช้ Entity Reference แทนตัวอักษรต้องห้าม

ตัวอักษร	รหัสที่ใช้แทน
&	&
<	<
>	>
'	'
"	"

2. ใช้ CDATA (Character DATA) หากเรามีตัวอักษรต้องห้ามจำนวนมากที่จำเป็นต้องแสดงผล อาจต้องแทนรหัสเป็นจำนวนมากจนทำให้ไม่น่าดูและอ่านไม่รู้เรื่องอีกด้วย ดังนั้นการกำหนด CDATA จะช่วยเราได้มาก

⁵ ขณะนี้เวอร์ชันล่าสุดของ XML คือ XML 1.1 (กำหนดโดย W3C โดยสามารถดูข้อกำหนดได้จาก <http://www.w3.org/TR/xml11/>) แต่ในที่นี้ จะอ้างอิงกฎต่าง ๆ ตาม XML 1.0 ซึ่งเป็นเวอร์ชันพื้นฐาน

การกำหนดส่วนที่เป็น CDATA คือการกำหนดให้ XML parser ไม่ทำการประมวลผลส่วนนั้น หากแต่ส่งข้อความทั้งหมดที่อยู่ในส่วนนั้นออกไปยังแอปพลิเคชัน โดยลักษณะของ CDATA ถูกแสดงในตัวอย่างที่ 2.15

ตัวอย่างที่ 2.15 แสดงตัวอย่างการใช้ CDATA

```
<comparison><![CDATA[6 is < 7 & 7 > 6]]></comparison>
```

ทุก ๆ สิ่งหลังจาก <![CDATA[จนไปถึง]]> จะถูกมองข้ามโดย XML parser และส่งข้อความทั้งหมดผ่านไปยังแอปพลิเคชันโดยไม่มีการประมวลผลใด ๆ ทั้งสิ้น

2.6 การใช้งาน Namespace

การใช้เนมสเปซ (namespace) ในเอกสาร XML เกิดขึ้นเพื่อแก้ปัญหาการใช้ชื่ออิลิเมนต์ของผู้สร้างเอกสาร XML ซึ่งสามารถตั้งได้อย่างอิสระ จนบางครั้งข้อมูลภายในอิลิเมนต์ที่มีชื่อหนึ่ง ๆ ในเอกสารฉบับหนึ่งอาจมีความหมายคนละเรื่องกับอิลิเมนต์ที่ชื่อเดียวกันนี้ในอีกเอกสารหนึ่ง ซึ่งถ้าเกิดมีการเชื่อมโยงระหว่างเอกสารทั้งสองจะทำให้เกิดความสับสน และค้นหาข้อมูลได้ไม่ตรงกับความต้องการ ซึ่งไม่ตรงกับจุดประสงค์ของ XML ที่ใช้อธิบายความหมายของข้อมูล

การกำหนดเนมสเปซทำได้โดยการเพิ่มคำนำหน้าชื่อ เพื่อแบ่งประเภทให้กับอิลิเมนต์ที่เป็นพวกเดียวกันหรือมาจากแหล่งเดียวกันให้เป็นกลุ่มเดียวกัน ซึ่งถ้าคำนำหน้านั้นซ้ำกันอีก การใช้เนมสเปซก็ จะไม่มีความหมาย เราจึงใช้เนมสเปซจากโดเมนเนมบนอินเทอร์เน็ต ซึ่งแต่ละบริษัทย่อมไม่มีโดเมนเนมที่ซ้ำกันแน่นอน ซึ่งการใช้เนมสเปซใน XML สามารถกำหนดได้ดังตัวอย่าง

ตัวอย่างที่ 2.16 แสดงตัวอย่างการใช้เนมสเปซ

```
<pers:person xmlns:pers="http://sernaferna.com/pers">
  <pers:name>
    <pers:title>sir</pers:title>
  </pers:name>
</pers:person>
```

จากตัวอย่างที่ 2.16 เรียกอีลิเมนต์ที่ถูกกำหนดคำว่า QName (qualified name) ซึ่งประกอบด้วย ส่วนที่เฉพาะ (local part) ซึ่งเป็นส่วนของชื่อจริง ๆ ของอีลิเมนต์ เช่น person, name, title และส่วนของคำนำหน้าเนมสเปซ (namespace prefix) ซึ่งจะระบุว่าชื่อดังกล่าวขึ้นอยู่กับเนมสเปซใด ซึ่งในที่นี้คือ pers:

ดังที่กล่าวไปแล้วว่า เพื่อไม่ให้เกิดปัญหาจากการที่เนมสเปซมีชื่อที่ซ้ำกัน เราจึงใช้โดเมนเนมในการกำหนดเนมสเปซ แต่โดเมนเนมนั้นยาวแล้วยุ่งยากในการใช้กับเอกสาร XML เราจึงใช้แอตตริบิวต์ **xmlns** เข้ามาช่วยเพื่อกำหนดให้ชื่อเนมสเปซ pers มีค่าเป็น <http://sernaferna.com/pers> นั้นเอง โดยหากเรามีหลายเนมสเปซก็สามารถระบุเพิ่มไปได้โดยวิธีนี้เช่นกัน ดังตัวอย่างที่ 2.17

ตัวอย่างที่ 2.17 การกำหนดเนมสเปซที่มีมากกว่า 1 เนมสเปซ

```
<pers:person xmlns:pers="http://sernaferna.com/pers"
             xmlns:xhtml="http://www.w3.org/1999/xhtml">
```

จากตัวอย่าง 2.17 ทำให้เห็นว่าเป็นการยุ่งยากที่จะเขียนเนมสเปซนำหน้าชื่อทุก ๆ อีลิเมนต์ในเอกสาร (แม้ว่าจะย่อชื่อโดเมนเนมเป็นชื่ออื่นแล้วก็ตาม) เพื่อแก้ปัญหาดังกล่าว เราจึงควรทำความรู้จักกับ **ดีฟอลต์เนมสเปซ** ซึ่งทำหน้าที่คล้ายคลึงกับเนมสเปซทั่วไป เพียงแต่ไม่ต้องระบุคำนำหน้าสำหรับทุก ๆ อีลิเมนต์ที่ใช้มัน ซึ่งแสดงในตัวอย่างที่ 2.18

ตัวอย่างที่ 2.18 การกำหนดดีฟอลต์เนมสเปซ

```
<person xmlns="http://sernaferna.com/pers">
  <name>
    <title>sir</title>
  </name>
</person>
```

จะเห็นว่าแอตตริบิวต์ xmlns นั้นไม่ได้ระบุคำนำหน้าเพื่อใช้กับเนมสเปซนี้อีกต่อไป เพราะมันเป็นการกำหนดดีฟอลต์เนมสเปซ ซึ่งอีลิเมนต์นี้และอีลิเมนต์ลูกหลานใด ๆ ที่ขึ้นอยู่กับอีลิเมนต์ **<person>** จะถือว่าอยู่ภายใต้เนมสเปซที่กำหนดเป็นดีฟอลต์ทั้งหมด (มีค่าเท่ากับเขียนแบบตัวอย่างที่ 2.16)

เรายังคงสามารถประกาศเนมสเปซได้มากกว่า 1 เนมสเปซ แต่สำหรับหนึ่งอิลิเมนต์ จะมีเพียงหนึ่งเนมสเปซเท่านั้นที่สามารถกำหนดให้เป็นดีฟอลต์ได้ ซึ่งจะอนุญาตให้เรากำหนด XML ได้ดังนี้

ตัวอย่างที่ 2.19 การกำหนดเนมสเปซอื่นเพิ่มจากดีฟอลต์เนมสเปซ

```
<person xmlns="http://sernaferna.com/pers"
        xmlns:xhtml="http://www.w3.org/1999/xhtml">
  <name/>
  <xhtml:p>This is XHTML</xhtml:p>
</person>
```

ในกรณีนี้ทุก ๆ อิลิเมนต์จะขึ้นอยู่กับเนมสเปซ `http://sernaferna.com/pers` ยกเว้นอิลิเมนต์ `<p>` ซึ่งเป็นส่วนหนึ่งของ XHTML เนมสเปซ (เมื่อเราประกาศเนมสเปซและคำนำหน้าต่าง ๆ ไว้ในอิลิเมนต์รากแล้ว อิลิเมนต์ลูกหลานก็มีสิทธิที่จะใช้คำนำหน้าเหล่านั้นได้)

2.7 XML ที่ถูกกฎพื้นฐาน (Basic Valid XML)

เอกสาร XML ที่ถูกกฎ (valid XML) คือ เอกสารที่ลงตัว⁶ และเป็นไปตามข้อกำหนดที่ระบุไว้ ในเอกสารที่สร้างขึ้นเพื่อตรวจสอบความถูกต้องของข้อมูลนั้น ซึ่งการตรวจสอบความถูกต้องของข้อมูลใน XML มี 2 วิธี คือ

- DTD (Document Type Definition)
- XML Data Schema

ทั้ง DTD และ XML Schema เป็นข้อตกลงที่เรากำหนดขึ้นเอง เพื่อใช้ตรวจสอบและเป็นข้อบังคับให้ข้อมูลต่าง ๆ ที่จัดเก็บในแต่ละ Tag แต่ละ Element มีความถูกต้องและเป็นไปตามที่เราต้องการ ซึ่งมีประโยชน์ต่อความเชื่อถือได้ของการแลกเปลี่ยนข้อมูล

ในความเป็นจริงแล้วมีเพียง DTD/Schema เดียวเท่านั้นที่ได้รับอนุญาตให้สัมพันธ์กับเอกสาร XML หนึ่งเอกสารที่กำหนด แต่โดยการตรวจสอบข้อมูลของเอกสาร XML ที่ว่า อาจแบ่งได้เป็น 2 ส่วนนั่นคือ ประกาศ DTD หรือ Schema ไว้ภายในเอกสาร XML เลย (เรียก **สับเซต**

⁶ **เอกสาร XML ที่ลงตัว** (Well-Formed XML) คือเอกสาร XML ที่เป็นไปตามไวยากรณ์และกฎโครงสร้างพื้นฐานของข้อกำหนด XML 1.0 (ซึ่งข้อกำหนดที่ควรทราบ ระบุไว้ในหัวข้อที่ 23 การสร้างเอกสาร Well-Formed XML) และสามารถศึกษาเพิ่มเติมได้จาก <http://www.w3.org> ส่วน **เอกสารที่ถูกกฎ** คือเอกสารที่ลงตัวและเป็นไปตามข้อกำหนดใน DTD หรือ Schema ด้วย

ภายใน) หรือทำการเชื่อมโยงเอกสาร XML นั้นกับเอกสาร DTD หรือ Schema ภายนอกก็ได้ (เรียก **สับเซตภายนอก**)

โดยการประกาศ DTD ในสับเซตภายในถือว่ามีนัยสำคัญ (priority) เหนือกว่าการประกาศ DTD ใน สับเซตภายนอก ดังนั้น เมื่อเราต้องการใช้มาตรฐาน DTD ที่มีอยู่แล้วแต่จะเปลี่ยนแปลงมันไปเล็กน้อยเพื่อวัตถุประสงค์ของเราเอง เราก็สามารถที่จะลิงค์ไปยัง DTD ในสับเซตภายนอกที่สามารถให้คำอธิบายและตรวจสอบความถูกต้องของข้อมูลส่วนมากในเอกสาร และประกาศ DTD ภายในเพิ่มเติม ซึ่งอาจจะเพื่อเพิ่มอิลิเมนต์หรือแอตทริบิวต์ใหม่ ๆ หรือเขียนทับประกาศส่วนที่เราอยากแก้ไขจาก DTD ภายนอกลงไปแทนได้ (เนื่องจากสับเซตภายในมีความสำคัญกว่า)

แต่สับเซตภายในก็มีข้อเสีย เนื่องจากความสามารถที่ประกาศทับ DTD ภายนอก ซึ่งถ้ามีใครคนหนึ่งแชร์เอกสารและ DTD กับเรา แล้วคนคนนั้นอาจเลือกที่จะเขียนทับข้อบังคับบางอย่างใน DTD ต้นฉบับของเรา แล้วมันก็จะทำให้เกิดปัญหาถ้าเอกสารนั้นถูกส่งกลับมาโดยที่ไม่มีส่วนเปลี่ยนแปลงสับเซตภายในใหม่ ๆ อยู่ด้วย จะเห็นได้ว่า สับเซตภายในยังมีประโยชน์ในระหว่างการพัฒนา DTD หรือเอกสารเดี่ยว ๆ ทั่วไป แต่มันจะมีประโยชน์น้อยถ้าเอกสารนั้นถูกแลกเปลี่ยนกับบุคคลหรือแอปพลิเคชันอื่น ๆ

เมื่อต้องการใช้ DTD/Schema ภายนอก เราจำเป็นต้องทำการลิงค์ไปยังเอกสาร DTD นั้น ด้วยการใช้มาร์คอัพที่เรียกว่า **Document Type Declaration** หรือเรียกว่า **การประกาศ DOCTYPE** (ไม่ได้ย่อเป็น DTD) ในการเชื่อมความสัมพันธ์ระหว่างเอกสารดังกล่าว โดยเอกสาร XML แต่ละฉบับสามารถถูกเชื่อมความสัมพันธ์เข้ากับเอกสารอีกเพียง 1 ฉบับเท่านั้น

`<!DOCTYPE document_element source location1 location2 [internal subset of DTD]>`

การประกาศ DOCTYPE ประกอบไปด้วย

- ตัวคั่น tag XML ทั่วไป (สัญลักษณ์ "<" และ ">")
- เครื่องหมายอัศเจรีย์ "!" บอกว่าเป็นการประกาศพิเศษ
- คีย์เวิร์ด DOCTYPE
- document_element : ชื่อของอิลิเมนต์เอกสาร หมายถึง อิลิเมนต์ที่เป็น Root Element ของเอกสาร XML นั้น (ต้องเป็นชื่อรูทอิลิเมนต์เท่านั้น ซึ่งจะทำให้ลูกหลานของมันทั้งหมดจะได้รับการสืบทอด DTD ไปใช้ด้วย)
- source : อาจเป็นคีย์เวิร์ด SYSTEM หรือ PUBLIC เพื่อบอกประเภท source ของเอกสาร DTD ที่อ้างถึง
- location1 location2 : ที่อยู่ของ DTD หนึ่งหรือสองตำแหน่ง ขึ้นอยู่กับค่าของ source
- [internal subset of DTD] : การประกาศเพิ่มเติมที่อ้างถึงสับเซตภายในของ DTD

สำหรับ ความหมายของคีย์เวิร์ดในส่วนของ source นั้น คีย์เวิร์ด **SYSTEM** ใช้เมื่อเราระบุตำแหน่งของ DTD อย่างเฉพาะเจาะจงด้วยการใช้ Uniform Resource Location (URL) หรือ Uniform Resource Identifier (URI) ดังตัวอย่างที่ 2.20

ตัวอย่างที่ 2.20 การกำหนด source ของ DTD โดยการใช้คีย์เวิร์ด SYSTEM

```
<!DOCTYPE BookCatalog SYSTEM "http://www.wrox.com/DTDs/BookCatalog.dtd" >
```

จากตัวอย่างที่ 2.20 parser จะทำการดึงเอา DTD ชื่อ BookCatalog.dtd จาก URL ที่ระบุจากเว็บไซต์ของ Wrox ถ้าไม่พบ DTD ดังกล่าว จะมีข้อผิดพลาดรายงานออกมาและ parser จะไม่สามารถตรวจสอบความถูกต้องของเอกสารได้

หากเอกสารอยู่บนเว็บเซิร์ฟเวอร์เดียวกันกับ DTD ก็สามารถเข้าถึง DTD ได้โดยตรงโดยไม่ต้องใช้แอดเดรสภายนอก ดังตัวอย่างที่ 2.21

ตัวอย่างที่ 2.21 การใช้คีย์เวิร์ด SYSTEM เข้าถึง DTD ที่อยู่บนเซิร์ฟเวอร์เดียวกับเอกสาร XML

```
<!DOCTYPE BookCatalog SYSTEM "file:///DTDs/BookCatalog.dtd">
```

อย่างไรก็ดี DTD ดังกล่าวก็มีความเป็นไปได้ที่อาจมีสำเนาอยู่ในหลาย ๆ ที่ ดังนั้นจะเป็นการดีกว่าถ้าแอปพลิเคชันที่ประมวลผลสามารถเรียกค้น DTD ที่สัมพันธ์กัน ด้วยการค้นหาของตัวมันเอง โดยการใช้คีย์เวิร์ด PUBLIC เข้ามาช่วย

คีย์เวิร์ด PUBLIC จะอนุญาตให้มีการอ้างถึง DTD โดย URL ที่ไม่เฉพาะเจาะจง ซึ่งสามารถอธิบายได้ด้วยตัวอย่างที่ 2.22

ตัวอย่างที่ 2.22 แสดงการกำหนด source ของ DTD โดยการใช้คีย์เวิร์ด PUBLIC

```
<!DOCTYPE BookCatalog PUBLIC "PublishingConsortium/BookCatalog"
"http://www.wrox.com/DTDs/BookCatalog.dtd">
```

จากตัวอย่างแอปพลิเคชัน XML จะมีความยืดหยุ่นมากกว่า โดยสามารถค้นหา DTD ที่แชร์ไว้ในเน็ตเวิร์กของ “สมาคมสิ่งพิมพ์” ได้จากส่วนของ “PublishingConsortium/BookCatalog”

ซึ่งถ้าparser หรือแอปพลิเคชันไม่สามารถค้นหา DTD จากตำแหน่งแรก (primary) ได้ ก็จะต้องเอา DTD จาก ตำแหน่งที่สอง⁷ (secondary) แทน

2.7.1 การตรวจสอบความถูกต้องของเอกสารด้วย DTD

Document Type Definition หรือ DTD ใช้เพื่ออธิบายและตรวจสอบความถูกต้องของ โครงสร้างและไวยากรณ์ภาษาของเอกสาร XML โดยจะระบุถึงข้อกำหนดสำหรับเอกสาร บัญญัติศัพท์ คำอธิบายโครงสร้าง รวมทั้งข้อห้ามต่าง ๆ สำหรับเอกสาร XML ที่เราเขียนขึ้น ทั้งนี้ เพื่อให้ผู้ที่นำเอกสาร XML ของเราไปใช้จะได้สามารถทำความเข้าใจและพัฒนาต่อได้อย่างถูกต้อง รวมทั้งเป็นบรรทัดฐานเพื่อใช้ตรวจสอบความถูกต้องของเอกสารที่เขียนขึ้นอีกด้วย ในความเป็นจริงแล้ว มีเพียง DTD เดียวเท่านั้นที่ได้รับอนุญาตให้สัมพันธ์กับเอกสาร XML หนึ่งเอกสารที่กำหนดไว้

การประกาศ DTD ทั้งหมดนั้นถูกค้นด้วยตัวค้นทั่วทั้งไป (“<” และ “>”) และต้องถูกระบุด้วยเครื่องหมายอัศเจรีย์ (“!”) ตามด้วยคีย์เวิร์ดหนึ่งคำและพารามิเตอร์เฉพาะของคีย์เวิร์ดที่ใช้ ดังนี้

<!keyword paramiter1 paramiter2 ... paramiterN>

ตารางที่ 2.2 คีย์เวิร์ดพื้นฐานที่ใช้ประกาศใน DTD

คีย์เวิร์ด	คำอธิบาย
ELEMENT	ประกาศชื่อชนิดอิลิเมนต์ XML และอิลิเมนต์ลูกที่ยอมรับให้ใช้ได้
ATTLIST	ประกาศชื่อแอตตริบิวต์ อิลิเมนต์ XML กับค่าแอตตริบิวต์ที่ใช้ได้ และ/หรือค่า ดีฟอลต์
ENTITY	ประกาศอักขระอ้างอิงพิเศษ, เท็กซ์มาโคร (คล้ายคลึงกับ #define ของภาษา C/C++) และเนื้อหาที่ใช้ซ้ำอื่น ๆ จากแหล่งข้อมูลภายนอก (เหมือน #include ในภาษา C/C++)
NOTATION	ประกาศเนื้อหา non-XML ภายนอก (เช่น ข้อมูลไบนารีอิมเมจ) และ แอปพลิเคชันภายนอกที่จัดการเนื้อหาดังกล่าว

⁷ ในตำแหน่งที่ 2 ทำหน้าที่เสมือนตำแหน่งสำรอง โดยต้องระบุเป็น URL แบบ SYSTEM เสมอแต่เมื่อนำมาใช้ร่วมกับ PUBLIC แล้วไม่ต้องประกาศคำว่า SYSTEM ซ้ำ

จากตารางที่ 2.2 คีย์เวิร์ดที่ใช้บ่อยมี 3 ตัว ได้แก่ การประกาศค่าอิลิเมนต์, การประกาศค่าแอตทริบิวต์ และการประกาศค่าเอนทิตี (entity) โดยมีรายละเอียดดังนี้

1. การประกาศค่าอิลิเมนต์

รูปแบบของการประกาศค่าอิลิเมนต์เป็น ดังนี้

<!ELEMENT ชื่ออิลิเมนต์ (เนื้อหาภายในอิลิเมนต์)>

เนื้อหาภายในอิลิเมนต์หนึ่งๆ มีโอกาสเป็นไปได้ 3 ลักษณะ คือ อาจจะเป็นอิลิเมนต์อื่นๆ อาจจะเป็นข้อความปกติ หรืออาจจะมีอะไรอยู่เลย (เป็นแท็กว่าง) หรือบางครั้งอาจจะมีอิลิเมนต์ปนอยู่กับข้อความปกติก็ได้ ไม่ผิดกติกาแต่อย่างใด ซึ่งจะยกตัวอย่างแต่ละลักษณะ ดังนี้

- แบบมีอิลิเมนต์อื่นๆ อยู่ภายใน เช่น

```
<picture>
  <width>300</width>
  <height>200</height>
</picture>
```

จากตัวอย่าง อิลิเมนต์ <picture> มีอิลิเมนต์ <width> และ <height> อยู่ภายใน

- แบบที่มีข้อความอยู่ภายใน เช่น

```
<titleการบริการตอบคำถามและช่วยการค้นคว้า </title>
```

- แบบไม่มีอะไรอยู่เลย หรือแท็กว่าง (empty tag) เช่น <linkto />

- แบบที่มีอิลิเมนต์ปนอยู่กับข้อความ เรียกว่า 'mixed content' เช่น

```
<detail>
  <subject>แนะนำการใช้คอมพิวเตอร์สืบค้นสารสนเทศ</subject>
  เป็นการแนะนำการใช้คอมพิวเตอร์ ในการ สืบค้นสารสนเทศภายใน
  สถาบันฯ และภายนอกสถาบันฯ
</detail>
```

สำหรับวิธีประกาศ DTD เพื่อตรวจสอบความถูกต้องของข้อมูลมีหลักดังนี้

1.1 ระบุ ANY หมายถึง มีอะไรภายในอิลิเมนต์ก็ได้ นั่นคือ ไม่บังคับว่าข้างในอิลิเมนต์เป็นอะไร โดยประกาศตามรูปแบบ ดังนี้

<!ELEMENT ชื่ออิลิเมนต์ ANY>

1.2 ระบุ PCDATA กรณีที่ต้องการบังคับให้เนื้อหาภายในอิลิเมนต์เป็นข้อความธรรมดา (เป็น PCDATA) เท่านั้น ประกาศตามรูปแบบ ดังนี้

<!ELEMENT ชื่ออิลิเมนต์ (#PCDATA)>

โดยPCDATA ย่อมาจาก Parsed Character Data หมายความว่า เนื้อหาส่วนที่เป็นข้อความนี้ เอาไว้ให้โปรแกรม XML Parser อ่าน เพื่อประมวลผล

1.3 ระบุ EMPTY เพื่อกำหนดให้อิลิเมนต์นั้นเป็นอิลิเมนต์ว่าง ประกาศตามรูปแบบ ดังนี้

<!ELEMENT ชื่ออิลิเมนต์ EMPTY >

1.4 ในกรณีที่มีอิลิเมนต์มีเนื้อหาภายในอิลิเมนต์หลากหลายแบบ เราสามารถใช้เครื่องหมายจุลภาค (,) หมายถึง “และ” ในกรณีที่มีหลายอิลิเมนต์ภายใต้อิลิเมนต์หลัก หรือใช้ไปป์ (|) ซึ่งหมายถึง “หรือ” ในกรณีที่ให้เลือกอย่างใดอย่างหนึ่ง เข้ามาช่วย พิจารณาตัวอย่างที่ 2.23

ตัวอย่างที่ 2.23 ตัวอย่างนิยาม DTD สำหรับอิลิเมนต์ <PersonName>

<!ELEMENT PersonName (SingleName | (FristName, MiddleName, LastName)) >

จากตัวอย่าง เป็นการกำหนดว่าอิลิเมนต์ <PersonName> ต้องประกอบด้วยอิลิเมนต์ลูกที่มีชื่อว่า <SingleName> เพียง 1 ตัว หรืออิลิเมนต์ <FristName>, <MiddleName>, <LastName> อย่างละ 1 ตัว ซึ่งต้องมีลำดับเรียงตามนี้ด้วย (การใช้เครื่องหมายจุลภาค ต้องเรียงลำดับตามนั้นเสมอ)

1.5 ใน XML มีเครื่องหมายพิเศษ 3 แบบ คือ +, * และ ? ให้เราพลิกแพลงใช้เพื่อเพิ่มลูกเล่นในการประกาศอิลิเมนต์โดยการระบุเครื่องหมายตามหลังอิลิเมนต์

- ระบุ + หมายความว่า จะต้องมียิลิเมนต์นั้นตั้งแต่ 1 อิลิเมนต์ขึ้นไป (คือ 1,2,3,...)

- ระบุ * หมายความว่า จะต้องมียิลิเมนต์นั้นกี่อิลิเมนต์ก็ได้ หรือไม่มีเลยก็ได้เช่นกัน (คือ 0,1,2,3,...)

- ระบุ ? หมายความว่า จะต้องไม่มีอิลิเมนต์ใด ๆ หรือ ถ้ามี จะมีอิลิเมนต์นั้นเพียง 1 อิลิเมนต์เท่านั้น (คือ 0,1)

1.6 หากเนื้อหาภายในอิลิเมนต์ มีทั้งอิลิเมนต์และข้อความ สามารถประกาศแบบผสมได้ ดังนี้

<!ELEMENT ชื่ออีลีเมนต์ (#PCDATA,อีลีเมนต์1,อีลีเมนต์2) >

โดย ในกรณีนี้มีข้อยกเว้นคือ เนื้อหาภายในไม่จำเป็นต้องเรียงลำดับก็ได้ ถึงแม้จะมีเครื่องหมายจุลภาคก็ตาม และข้อควรทราบอีกอย่างคือ จริง ๆ แล้ว เราสามารถใช้คีย์เวิร์ด ANY แทนก็ได้ ซึ่งความหมายไม่ได้แตกต่างกันเลย

2. การประกาศค่าแอตทริบิวต์

การประกาศค่าแอตทริบิวต์เป็นส่วนที่สำคัญเช่นกัน รูปแบบการประกาศแอตทริบิวต์มีดังนี้

<!ATTLIST elementName attrName attrType attrDefault defaultValue>

- **elementName** ชื่อของอีลีเมนต์ที่ต้องการกำหนด
- **attrName** ชื่อแอตทริบิวต์ที่อนุญาตให้ใช้ได้สำหรับอีลีเมนต์นี้
- **attrType** ชนิดของแอตทริบิวต์ โดยภายใต้ประกาศข้อแนะนำของ XML 1.0 ได้กำหนดชนิดของแอตทริบิวต์ไว้ 10 ชนิด ดังแสดงในตารางที่ 2.3
- **attrDefault** มีไว้เพื่อสั่งการว่าจำเป็นต้องมีแอตทริบิวต์อยู่หรือไม่ และถ้าไม่ parser ตรวจสอบจะจัดการกับการที่ไม่มีอยู่ในเอกสารได้อย่างไร โดย Attribute Default มี 4 ชนิด ดังแสดงในตารางที่ 2.4
- **defaultValue** ค่าดีฟอลต์ สำหรับอีลีเมนต์ที่ไม่ปรากฏแอตทริบิวต์ในเอกสาร parser อาจจะใส่ค่าดีฟอลต์นี้มาให้แทน

ตารางที่ 2.3 ชนิดของแอตทริบิวต์ สำหรับการประกาศ ATTLIST

ชนิดของแอตทริบิวต์	คำอธิบาย
CDATA	เป็นตัวอักษรหรือข้อความปกติ สิ่งที่จะต้องระวังคือ ข้อความที่กำหนดเป็นค่าแอตทริบิวต์นั้นต้องไม่มีอักขระที่สงวนไว้ ซึ่งได้แก่ เครื่องหมายน้อยกว่า (<), เครื่องหมายมากกว่า (>), เครื่องหมายคำพูด double quote ("), เครื่องหมายคำพูด single quote (') และ เครื่องหมาย ampersand (&) ถ้าจำเป็นต้องมีก็ต้องระบุเป็น Entity Reference แทน คือ < > " ' และ &
ค่าจระไน (enumerated)	ค่าของแอตทริบิวต์ต้องเป็นหนึ่งในหลายๆค่าเหล่านี้ที่ถูกกำหนดเชิงประจักษ์อยู่ใน DTD หรือลิสต์ตัวเลือก

ตารางที่ 2.3 (ต่อ) ชนิดของแอตตริบิวต์ สำหรับการประกาศ ATTLIST

ชนิดของแอตตริบิวต์	คำอธิบาย
ID	ค่าของแอตตริบิวต์ที่เป็นตัวบ่งชี้ที่ unique สำหรับอินสแตนซ์ของอิลิเมนต์ชนิดนี้ ต้องเป็น text สตริงที่สอดคล้องกับกฎการตั้งชื่อของ XML และอักขระตัวแรกเป็นได้แค่ตัวอักษรหรือขีดเส้นใต้ (_) หรือโคลอน (:) เท่านั้น
IDREF	การอ้างถึงอิลิเมนต์ด้วยแอตตริบิวต์ ID ที่มีค่าเดียวกับค่าของ IDREF
IDREFS	ลิสต์ของ IDREF ที่คั่นด้วย white space
NMTOKEN	เนมโทเคน ⁸ เป็นสตริงที่สอดคล้องกับกฎการตั้งชื่อ XML ยกเว้นอักขระตัวแรกของชื่อ อาจเป็นตัวอักษรกำหนดชื่อที่ถูกกฎใด ๆ ก็ได้
NMTOKENS	ลิสต์ของ NMTOKEN ที่คั่นด้วย white space
ENTITY	ชื่อของเอนทิตีที่ถูกกำหนดไว้ล่วงหน้า (pre-defined)
ENTITIES	ลิสต์ของ ENTITY ที่คั่นด้วย white space
NOTATION	ค่าแอตตริบิวต์ต้องเป็น notation type ที่ได้รับการประกาศเชิงประจักษ์อยู่ที่ใด ๆ ใน DTD

ตารางที่ 2.4 ชนิดของ Attribute default

Attribute default	คำอธิบาย
#REQUIRED	แอตตริบิวต์นั้นต้องปรากฏอยู่ในทุก ๆ elementName นั้น
#IMPLIED	แอตตริบิวต์นั้น เป็นตัวเลือก
#FIXED (ตามด้วยค่าดีฟอลต์)	แอตตริบิวต์อาจปรากฏหรือไม่ปรากฏในเอกสารก็ได้ ถ้าแอตตริบิวต์นั้นปรากฏต้องมีค่าตรงกับค่าดีฟอลต์ ถ้ามันไม่ปรากฏ parser อาจผลิตค่าดีฟอลต์มาให้
ค่าดีฟอลต์	แอตตริบิวต์อาจปรากฏหรือไม่ปรากฏในเอกสารก็ได้ ถ้าแอตตริบิวต์นั้นปรากฏมันจะต้องเป็นค่าใด ๆ ที่ตรงกับการประกาศ ATTLIST เหล่านั้น ถ้ามันไม่ปรากฏ parser อาจผลิตค่าดีฟอลต์มาให้

⁸ โทเคน (Token) หมายถึง ส่วนประกอบของสายอักขระที่แบ่งแยกลักษณะได้เด่นชัด (สุวัฒนา สุขสมจินต์.

3. การประกาศค่า ENTITY

เอนทิตี (entity) จะใช้สำหรับอ้างถึง “ทรัพยากร” หรือข้อมูลที่เรานิยามไว้ก่อน ทำให้เราสามารถนำทรัพยากรหรือข้อมูลนั้นกลับมาใช้ใหม่ (reuse) โดยไม่ต้องเขียนบ่อยๆ การประกาศเอนทิตีใน DTD มีรูปแบบดังนี้

<!ENTITY ชื่อเอนทิตี ทรัพยากร>

ส่วนใหญ่แล้วทรัพยากรหรือข้อมูล ก็คือ ข้อความ (สตริง) นั่นเอง

ความสำคัญของ DTD (Document Type Definition) คือ

1. เป็นการกำหนดรูปแบบโครงสร้างของเอกสาร XML (หรือเรียกว่า Data Model) ว่าประกอบไปด้วยแท็กอะไรบ้าง, มีแอตทริบิวต์อะไรบ้าง, ค่าของแอตทริบิวต์เป็นอะไรได้บ้าง, ลักษณะการซ้อนแท็กเป็นอย่างไรได้บ้าง

2. เป็นสัญลักษณ์ระหว่างองค์กร ที่จะแลกเปลี่ยนข้อมูลกันในรูปของ XML เพื่อให้เอกสารที่สร้างขึ้นมามีรูปแบบตรงตามที่ตกลงกันได้ และจะได้อ่าน นำเอกสาร XML นั้นไปใช้งานอย่างถูกต้อง

โดยโครงสร้างของเอกสาร XML หลังจากสร้าง DTD แล้วจะมีลักษณะโครงสร้างเป็นลำดับชั้นเหมือนโครงสร้าง ต้นไม้

2.7.2 การตรวจสอบความถูกต้องของเอกสารด้วย XML Schema

สกีมา ถูกกำหนดไว้โดยทั่วไปว่าเป็น “การจัดองค์กรหรือโครงสร้างของฐานข้อมูล” มักจะสืบทอดคุณสมบัติมาจากโมเดลข้อมูล โครงสร้างดังกล่าวจะได้รับการอธิบายด้วยการใช้บัญญัติศัพท์ในการ ควบคุมบางชนิด ที่ใช้ตั้งชื่อรายการของข้อมูลและลิสต์ข้อความควบคุมใด ๆ ที่เหมาะสม (ชนิดของ ข้อมูล, ค่าที่ถูกต้อง/ไม่ถูกต้อง, รูปแบบพิเศษ เป็นต้น)

2.7.2.1 การกำหนดสกีมา

การประกาศว่าส่วนใดเป็นสกีมานั้นสามารถประกาศได้โดยใช้อิลิเมนต์ <schema>⁹ หรือ <Schema>¹⁰ ในการประกาศ ดังตัวอย่างที่ 2.24

⁹ ตามข้อกำหนดของ XML schema อย่างเป็นทางการตามร่างของ W3C

¹⁰ สำหรับกำหนดสกีมาใน XDR ของไมโครซอฟท์ ซึ่งผลิตสกีมาของตนเองขึ้นมาใช้ โดยต้องตั้งชื่อให้สกีมาด้วยแอตทริบิวต์ name ด้วย ดังนั้นจึงต้องกำหนดในรูปแบบ <Schema name="MyXDR"> ... </Schema> เป็นต้น

ตัวอย่างที่ 2.24 การประกาศสกีมา

```
<schema ...>
    ...<!-- เนื้อหา -->
</schema>
```

สกีมาหนึ่ง ๆ อาจถูกรวมอยู่ในเอกสาร XML อื่น ๆ หรือแยกเป็นเอกสารที่เป็น XML สกีมาโดยเฉพาะเลยก็ได้ (.xdr) โดยปกติแล้วในการกำหนดสกีมามักจะกำหนดโดยระบุ namespace ด้วย ดังตัวอย่างที่ 2.25

ตัวอย่างที่ 2.25 การประกาศสกีมาที่มีเนมสเปซ

```
<xsd:schema xmlns:xsd="http://www.w3.org/1999/XMLSchema"
            xmlns:xsi="http://www.w3.org/1999/XMLSchema-instance">
    ...
</schema>
```

2.7.2.2 ชนิดของข้อมูล แบ่งออกเป็น 2 แบบ ได้แก่

1. **Simple Type Definition** กำหนดด้วยอิลิเมนต์ <simpleType> ใช้สำหรับประกาศข้อบังคับเกี่ยวกับ ชนิดของข้อมูล รูปแบบ ข้อบังคับ สำหรับให้ข้อมูลอื่น ๆ สืบทอดไปได้
2. **Complex Type Definition** กำหนดด้วยอิลิเมนต์ <complexType> ใช้สำหรับอธิบายโมเดลเนื้อหา อิลิเมนต์และแอตตริบิวต์ที่เกี่ยวกับรายละเอียดของข้อมูลในเชิงลึก

2.7.2.3 การประกาศอิลิเมนต์

การประกาศอิลิเมนต์ของสกีมา ใช้ <element> ซึ่งคล้ายคลึงกับการประกาศอิลิเมนต์ของ DTD เป็นอย่างมาก (นั่นคือ <!ELEMENT..>) แต่สามารถอธิบายโมเดลเนื้อหาที่สลับซับซ้อนได้ดีกว่า มีแอตตริบิวต์ที่สำคัญดังแสดงในตารางที่ 2.5

ตารางที่ 2.5 แอตตริบิวต์ที่สำคัญในการประกาศอิลิเมนต์สำหรับ XML สกีม่า

ชื่อแอตตริบิวต์	ชนิดข้อมูล	ค่าดีฟอลต์	คำอธิบาย
name	NCName	ไม่มี	ชื่ออิลิเมนต์ที่กำลังได้รับการประกาศ
ref	QName	ไม่มี	ชื่อ <element> ที่อ้างถึง
type	QName	ไม่มี	ใช้กำหนดชนิดข้อมูล
minOccurs	nonNegativeInteger	1	จำนวนอิลิเมนต์ลูก ต่ำสุด ที่มีได้
maxOccurs	nonNegativeInteger	minOccurs	จำนวนอิลิเมนต์ลูก สูงสุด ที่มีได้
default	string	ไม่มี	กำหนดค่าดีฟอลต์ของอิลิเมนต์
fixed	string	ไม่มี	กำหนดค่าคงที่ (ดีฟอลต์) ของอิลิเมนต์
id	ID	ไม่มี	ตัวบ่งชี้ สำหรับอิลิเมนต์

2.7.2.4 การประกาศแอตตริบิวต์

การประกาศสกีม่า <attribute> คล้ายคลึงกับการประกาศ <!ALLLIST ..> ใน DTD โดยการประกาศ <attribute> จะเชื่อมความสัมพันธ์ระหว่างชื่อแอตตริบิวต์กับ simple type ที่กำหนด และให้คำอธิบายเพื่อใช้ในการตรวจสอบความถูกต้อง กำหนดค่าดีฟอลต์ กำหนดจำนวนครั้งที่ปรากฏ สำหรับแอตตริบิวต์นั้น ๆ

DTD นั้นก็จัดเป็นรูปแบบหนึ่งของสกีม่า แต่ DTD ก็ยังมีข้อจำกัดบางประการที่ทำให้ DTD เป็นผู้ตรวจสอบไวยากรณ์ของเอกสารที่ด้อยกว่า XML Schema โดยความแตกต่างระหว่างการใช้งาน DTD และ XML Schema เพื่อตรวจสอบความถูกต้องของข้อมูล XML แสดงในตารางที่ 2.6

ตารางที่ 2.6 ความแตกต่างระหว่าง DTD และ XML Schema

	DTD	XML Schema
ไวยากรณ์	EBNF + XML เทียม	XML 1.0
นามสกุล	.dtd	.xdr
ตัวอย่างการประกาศ	<pre><!ELEMENT ElementName ... > <!ATTLIST ElementName ... ></pre>	<pre><schema ...> ...<!-- เนื้อหา --> </schema></pre>

ตารางที่ 2.6 (ต่อ) ความแตกต่างระหว่าง DTD และ XML Schema

	DTD	XML Schema
เครื่องมือ	ใช้เครื่องมือ SGML ใด ๆ ที่มีอยู่ แต่เครื่องมือเหล่านี้มีราคาแพง	ใช้เครื่องมือ XML ใด ๆ เกือบทั้งหมด รวมถึง DOM, XSLT และเบราเซอร์ที่รับรู้ XML และมีความสลับซับซ้อนกว่า
สนับสนุน DOM ¹¹	ไม่	สามารถแสดงผลและจัดการได้ด้วย DOM เพราะสกีมาใช้ไวยากรณ์ XML
โมเดลเนื้อหา	เสียเปรียบ – มีเพียงลิสต์ตัวเลือกหรือลิสต์ลำดับอย่างง่าย ไม่สามารถใช้เนื้อหาเหล่านี้ผสมกันได้; ระบุจำนวนครั้งที่ปรากฏอิลิเมนต์ได้เพียง 0, 1 หรือหลายครั้ง เท่านั้น; ไม่มีอิลิเมนต์ที่กำหนดชื่อได้หรือกลุ่มของแอตตริบิวต์ให้ใช้	ได้เปรียบ – มีรายละเอียดมากกว่า ไม่ถูกจำกัดด้วยเนื้อหาแบบผสม; สามารถกำหนดจำนวนครั้งที่อิลิเมนต์ปรากฏขึ้นจริงได้; มีกลุ่มของโมเดลที่กำหนดชื่อได้
การกำหนดชนิดข้อมูล	เสียเปรียบ – กำหนดสตริง เนมโทเคน ID และอื่น ๆ ได้เพียงเล็กน้อย	ได้เปรียบ – สามารถกำหนดชนิดข้อมูลสมัยใหม่ได้มาก เช่น สตริง ตัวเลข วัน เวลา ทำให้สามารถใช้ค้นหาข้อมูลที่เจาะจงชนิดของข้อมูลได้
ขอบเขตของชื่อ	แบบโกลบอลเท่านั้น	ได้ทั้งแบบโกลบอลและโลคอล
การสืบทอดคุณสมบัติ	ไม่มี	มี
ความสามารถในการขยายขนาด	จำกัด - เมื่อมีการเพิ่มประกาศอื่น ๆ หรือความสามารถของ XML ต้องไปแก้ไข DTD ด้วย	ไม่จำกัด

¹¹ Document Object Model (DOM) เป็น Application Programming Interface (API) ที่ทำงานร่วมกับ XML ที่ช่วยให้เราสร้างเอกสารและส่วนต่าง ๆ ของเอกสาร, ท่องไปตามเอกสาร, ย้าย, สำเนา, ลบส่วนต่าง ๆ, เพิ่มหรือแก้ไข แอตตริบิวต์ในเอกสาร XML ได้

ตารางที่ 2.6 (ต่อ) ความแตกต่างระหว่าง DTD และ XML Schema

	DTD	XML Schema
ข้อจำกัดที่สืบทอด	ใช่	ไม่
สนับสนุนหลาย ๆ บัญญัติศัพท์	ไม่ – หนึ่ง DTD ต่อเอกสารหนึ่งฉบับ	ใช่ – มีได้มากเท่าที่จำเป็น ขึ้น กับเนมสเปซใน XML
Dynamic	ไม่	ใช่ – สกีมามีสามารถถูกเลือกใช้ หรือ แก้ไขได้ในขณะที่รันอยู่ได้